

# PROGRAMAÇÃO WEB

## GUIÃO DO PROJETO

Os alunos devem desenvolver um **website dinâmico** para a gestão da **Clínica Veterinária “MiACaoMiGo - Clínica e Adoção Animal”**. Além do **frontend**, devem também criar uma **API de dados** para gerir os dados do sistema. **Será fornecido um esqueleto da API de dados que pode ser facilmente adaptado a cada projeto.**

## TECNOLOGIAS

Os alunos devem utilizar as seguintes tecnologias:

- **Frontend:** HTML, CSS, JavaScript e Bootstrap.
- **Backend/API:** JavaScript (Node.js + Express.js).
- **Base de Dados:** PostgreSQL (responsabilidade de outra unidade curricular).

## ESTRUTURA DO PROJETO

O projeto deve estar organizado da seguinte forma:

```
/projeto
├── /frontend
│   ├── /css                # Folhas de estilo
│   ├── /js                 # Scripts JavaScript
│   ├── /img                # Imagens do site
│   ├── /pages              # Páginas HTML adicionais
│   │   ├── login.html
│   │   ├── clientes.html
│   │   ├── animais.html
│   │   ├── consultas.html
│   │   └── adcooes.html
│   └── index.html          # Página principal
├── /backend
│   ├── app.js              # Configuração da aplicação Express
│   ├── server.js           # Arranque do servidor
│   ├── /config              # Configurações gerais
│   │   └── database.js      # Ligação à base de dados
│   ├── /models              # Modelos de dados
│   │   └── animalModel.js
│   ├── /controllers         # Lógica da aplicação
│   │   └── animalController.js
│   ├── /routes              # Rotas da API
│   │   └── animais.js
│   └── /middlewares         # Middleware (autenticação, validação, etc.)
│       └── authMiddleware.js
├── package.json            # Dependências do projeto
└── README.md               # Documentação do projeto
```

É de notar que os ficheiros apresentados na estrutura são meramente exemplos.

## TEMA E REQUISITOS DO PROJETO

O projeto de **Programação Web** terá por base o mesmo tema e os mesmos requisitos definidos nos guiões de projeto das unidades curriculares de **Bases de Dados (BD)** e **APS**. Desta forma, o objetivo do projeto consiste em **desenvolver um website que suporte todas as funcionalidades previamente definidas**.

Para a sua implementação, os estudantes deverão ter em consideração todos os conteúdos abordados nas aulas, nomeadamente no que diz respeito à **estrutura e sintaxe das tecnologias utilizadas**, bem como às **boas práticas de programação** e aos **princípios de segurança do sistema**.

### Tema:

A Clínica Veterinária “**MiACaoMiGo - Clínica e Adoção Animal**” pretende desenvolver um sistema de informação da clínica que seja multifuncional, cuja missão principal é o acolhimento de animais abandonados, a sua reabilitação, e posterior encaminhamento para adoção responsável.

Paralelamente, a clínica atua como espaço de prestação de cuidados veterinários e de apoio à comunidade, promovendo o bem-estar animal e a sensibilização social. O objetivo principal da clínica é prestar cuidados veterinários a animais acolhidos e a animais de clientes externos; criar um sistema organizado de acolhimento, acompanhamento e adoção de animais abandonados; disponibilizar a venda de produtos para animais (alimentação, higiene, acessórios, medicamentos autorizados); divulgar campanhas de sensibilização, adoção e esterilização; implementar um sistema de informações e notificações para utilizadores (eventos, campanhas, lembretes, novidades).

### Funcionalidades principais:

- Área Clínica: consultas, tratamentos, vacinação, esterilização e acompanhamento médico.
- Área de Acolhimento: espaços adequados para permanência temporária dos animais.
- Sistema de Adoção: perfis de animais disponíveis, processo de candidatura e acompanhamento pós-adoção.
- Loja de Produtos: catálogo, gestão de stock e vendas.
- Informações e Campanhas: divulgação de iniciativas, eventos e ações solidárias.
- Notificações: alertas sobre campanhas, novos animais para adoção, datas importantes e comunicações gerais.

O seu **público-alvo** são as pessoas interessadas em adotar animais; clientes da clínica veterinária; voluntários e colaboradores e a comunidade em geral sensibilizada para a causa animal.

### Principais Requisitos do Sistema:

- Interface intuitiva e acessível;
- Gestão eficiente de dados de animais, adoções e clientes; Sistema de notificações (e-mail, SMS ou app);
- Atualização fácil de campanhas e informações;

- Conformidade com normas de bem-estar animal e legislação aplicável.

### **Resultados Esperados**

- Aumento da taxa de adoção responsável.
- Melhoria na gestão dos animais acolhidos.
- Maior envolvimento da comunidade.
- Sustentabilidade financeira através da venda de produtos e serviços.
- Reforço da consciência social sobre o abandono animal.

### **Requisitos do Sistema**

1. A clínica tem diversos funcionários e veterinários cuja informação interna contempla:
  - um código,
  - nome completo,
  - morada,
  - email
  - NIF
  - contacto profissional,
  - sobre os veterinários é necessário também a informação sobre a especialidade (por exemplo, cirurgia, dermatologia, cardiologia).
2. Cada cliente (dono do animal) tem:
  - um número de cliente,
  - nome completo,
  - morada,
  - telefone,
  - email,
  - NIF.
3. Cada cliente pode ter um ou mais animais registados na clínica, para cada animal devem ser guardados:
  - um código do animal,
  - nome,
  - espécie (cão, gato, coelho, etc.),
  - raça,
  - data de nascimento,
  - sexo,
  - e um estado que indique se o animal é doméstico (pertence a um cliente) ou resgatado (aguarda adoção).
4. A clínica realiza consultas, em que:
  - cada consulta tem uma data, hora, motivo e observações,
  - é associada a um único animal e a um único veterinário,
  - pode gerar prescrições de medicamentos.
5. Cada medicamento possui:
  - um código de medicamento,
  - nome comercial,

- descrição,
  - dosagem recomendada.
6. O sistema deverá também permitir o registo das faturas associadas às consultas, contendo:
- um número de fatura,
  - data,
  - valor total,
  - e a referência à consulta correspondente.
7. Gestão de Animais Abandonados e Adoções:
- A clínica colabora com entidades de recolha (como associações ou municípios) que entregam animais abandonados. Para cada entidade, devem ser registados o nome, contacto e localização.
  - Cada animal resgatado tem associada a data de recolha, o local onde foi encontrado, o estado clínico inicial e a entidade que o apresentou.
  - Os animais resgatados podem ser adotados por clientes da clínica ou por novos adotantes. Para cada adoção, devem ser guardados:
    - a data de adoção,
    - o cliente/adotante,
    - o animal adotado,
    - e o responsável da clínica (veterinário ou funcionário) que supervisionou o processo.

### Autenticação e Perfis de Utilizador

- Três tipos de utilizadores: **Administrador, Veterinário e Funcionário**.

### Backend e API de dados

- **Adaptar** o esqueleto da **API de dados** fornecido a cada projeto.
- A API deve permitir:
  - **CRUD** (Create, Read, Update, Delete) de utilizadores, aulas, inscrições e dispositivos de monitorização.
  - **Endpoints** seguros com autenticação (**JWT** ou outra forma segura).
  - Retorno de dados em formato **JSON**.
  - **As passwords devem ser encriptadas antes de serem guardadas na base de dados.**

### Frontend e Integração com Bootstrap

- O site deve ser responsivo e visualmente organizado usando **Bootstrap**.
- Deve incluir **botões, formulários e tabelas** para melhor usabilidade.
- A comunicação com a **API de dados** será feita via **Fetch API (método JavaScript)**.
- O website deve incluir Login, Criar Conta, Página de Gestão de Adoções, entre outras.

### Boas Práticas no Código

- Todo código (HTML, JavaScript) deve conter **comentários explicativos**.
- As **funções** devem ter **nomes curtos e descritivos**, como `carregarUtilizadores()` em vez de `getDataFromAPI()`.
- Manter o código **organizado** seguindo a estrutura de pastas definida.

- As imagens devem estar dentro da pasta /img/.

## CONCEITOS ESSENCIAIS

**Bootstrap:** Bootstrap é um **framework CSS** que facilita a criação de interfaces responsivas e estilizadas. Ele oferece componentes prontos, como botões, formulários e tabelas, permitindo um desenvolvimento mais rápido e consistente.

**HTML:** Linguagem de marcação usada para estruturar páginas web.

**CSS:** Usado para estilizar os elementos HTML e tornar o site visualmente atraente.

**JavaScript:** Linguagem de programação que permite interatividade e manipulação dinâmica do conteúdo da página.

**API (Application Programming Interface):** Interface que permite a comunicação entre sistemas diferentes (neste caso entre a base de dados e o website).

**Node.js:** Ambiente de execução JavaScript no lado do servidor, permitindo criar APIs e aplicações escaláveis.

**Express.js:** Framework para **Node.js** que simplifica a criação de servidores e gerenciamento de rotas para APIs.

**CRUD (Create, Read, Update, Delete):** Operações básicas para manipulação de dados em um sistema.

**JWT:** Método seguro para autenticação, permitindo que os utilizadores façam login e mantenham a sessão ativa sem necessitar de reenviar credenciais a cada requisição.

**JSON:** Formato de troca de dados usado para armazenar e transmitir informações entre sistemas.

**Fetch API:** Funcionalidade do JavaScript que permite fazer requisições HTTP para obter ou enviar dados de um servidor de forma assíncrona.

## ENTREGA

- **Data final:** 23 de maio de 2026.
- **Entregáveis:** Relatório + Pasta do Projeto.
- O projeto deve ser submetido numa pasta a ele destinado no **OneDrive**.
- Trabalhos enviados após este prazo poderão **sofrer penalizações significativas na avaliação**, salvo casos devidamente justificados e aceites previamente.
- O uso de meios ou ferramentas para a realização do trabalho que **não tenham sido abordados em aula** poderá ter implicações e deverá ser devidamente documentado no relatório.
- **Apresentação pública e defesa (com PowerPoint):** 25 e 28 de maio de 2026.